

DOCKER COMMAND REFERENCE – COMPLETE PRACTICAL GUIDE –

A Comprehensive, Production-Grade
Systems Administrator & Engineer
Handbook

Systems Documentation Series • Production Edition

TABLE OF CONTENTS

SECTION 1: DOCKER CLI BASICS

`docker / docker help / docker version / docker info / docker context`

SECTION 2: IMAGE MANAGEMENT COMMANDS

`docker pull / push / search / images / rmi / inspect / history / save / load / tag / prune / import`

SECTION 3: CONTAINER LIFECYCLE MANAGEMENT

`docker run / create / start / stop / restart / pause / unpause / kill / wait / rm / rename / update / inspect / top / stats / logs / attach / exec / cp / diff / port / commit / export`

SECTION 4: BUILD COMMANDS

`docker build / builder / buildx`

SECTION 5: VOLUME ENGINE COMMANDS

`docker volume (create / ls / inspect / rm / prune)`

SECTION 6: NETWORK ARCHITECTURE COMMANDS

`docker network (create / connect / disconnect / inspect / ls / rm / prune)`

SECTION 7: SYSTEM LEVEL ENGINE COMMANDS

`docker system (df / events / info / prune)`

SECTION 8: DIAGNOSTICS, LOGGING & TROUBLESHOOTING

Engine Event Streams, Stream Capture, Live Profiling Patterns

SECTION 9: DOCKER COMPOSE COMMANDS REFERENCE

`up / down / build / pull / push / start / stop / restart / logs / exec / run / ps / config / images`

SECTION 10: SWARM ORCHESTRATION ENGINE

`init / join / leave / service create / update / rm / node ls / stack deploy / stack rm`

SECTION 11: REPOSITORIES & REGISTRY ENGINE

`login / logout / push / pull / tag / search`

SECTION 12: AGGRESSIVE DISK & RESOURCE CLEANUP

System Pruning, Resource Sweeps, Boundary Warnings

APPENDICES

Appendix A: Cheat Sheet • Appendix B: Flag Directory • Appendix C: Exit Codes •
Appendix D: Engine Troubleshooting Matrix • Appendix E: Master Lookup Table

SECTION 1: DOCKER CLI BASICS

Command: `docker`

Purpose: The base execution command for the Docker Command Line Interface. It initiates all interactions with the Docker Engine daemon over the REST API layer.

Syntax:

```
docker [OPTIONS] COMMAND [ARG...]
```

Options Table:

Option	Description
<code>-D, --debug</code>	Enable debug mode to output highly verbose execution logs from the CLI client and API interaction frames.
<code>-H, --host</code>	Daemon socket(s) to connect to. Specifies the Unix domain socket or TCP port bindings (e.g., <code>unix:///var/run/docker.sock</code> or <code>tcp://127.0.0.1:2375</code>).
<code>-l, --log-level</code>	Set the logging level explicitly ('debug', 'info', 'warn', 'error', 'fatal').
<code>--tls</code>	Enable TLS transport layer verification security protocol; implied by <code>--tlsverify</code> .
<code>--tlsverify</code>	Use TLS certificates and strictly verify the remote daemon's authenticity before parsing payloads.

Common Usage Patterns:

- Execute standard daemon query with debug flags enabled.
- Direct a remote docker daemon instance over a secure TCP interface socket endpoint structure.

Real-World Examples:

```
$ docker -D info
```

```
$ docker -H tcp://192.168.1.100:2376 --tlsverify info
```

Common Mistakes / Edge Cases:

Executing commands against a remote host without exposing or configuring the corresponding certificate directories, resulting in TLS layer validation handshaking structural lockouts.

Related Commands: `docker info`, `docker context`

Command: `docker help`

Purpose: Retrieves comprehensive contextual text instructions, argument lists, operational definitions, and runtime summaries for any executable structural entity inside the CLI infrastructure framework.

Syntax:

```
docker help [COMMAND]
```

Common Usage Patterns:

- Query system help.
- Query specific help blocks.

Real-World Examples:

```
$ docker help
```

```
$ docker help run
```

```
$ docker run --help
```

Common Mistakes / Edge Cases:

Misunderstanding options placement; `--help` works universally at the trailing boundary of strings.

Related Commands: `docker info`

Command: `docker version`

Purpose: Shows precise configuration layouts, semantic version strings, Git hashes, API build baselines, execution architectures, OS layers, and runtime binaries for both client context and remote server engines.

Syntax:

```
docker version [OPTIONS]
```

Options Table:

Option	Description
<code>-f, --format</code>	Format the system engine output structure utilizing Go templates syntax engine expressions.

Common Usage Patterns:

- Dump version layouts.
- Extract precise API target parameters.

Real-World Examples:

```
$ docker version
```

```
$ docker version --format '{{.Server.Version}}'
```

Common Mistakes / Edge Cases:
Assuming identical capabilities across environments without checking the API baseline version numbers.

Related Commands: `docker info`

Command: `docker info`

Purpose: Displays critical cluster wide parameters, driver states, compute layouts, kernel variables, container counts, file storage backends, and logging infrastructures.

Syntax:

```
docker info [OPTIONS]
```

Options Table:

Option	Description
<code>-f, --format</code>	Format the system engine output structure utilizing Go templates syntax engine expressions.

Common Usage Patterns:

- Review full engine storage properties.
- Query storage driver configurations directly.

Real-World Examples:

```
$ docker info
```

```
$ docker info --format '{{.Driver}}'
```

Common Mistakes / Edge Cases:

Overlooking output indicators regarding insufficient configuration parameters like kernel cgroups configurations missing memory swaps.

Related Commands: `docker version`, `docker system df`

Command: `docker context`

Purpose: Provides advanced tooling to efficiently define, manage, switch, and maintain communication configurations targeting distinct isolation daemons, Kubernetes engines, or cloud swarms.

Syntax:

```
docker context [COMMAND]
```

Options Table:

Option	Description
<code>ls</code>	List all configured docker daemon endpoint contexts structural entries.
<code>create</code>	Instantiate a new structural descriptor referencing specific daemon endpoint targets.
<code>use</code>	Instally transition active working environments to process commands via specified engine target references.
<code>inspect</code>	Display structural nested configurations pertaining to explicit context identifiers.

Common Usage Patterns:

- Isolate stage and production environments configuration states securely.
- Transition global execution targets.

Real-World Examples:

```
$ docker context ls
```

```
$ docker context create prod-env --docker host=tcp://prod.engine.internal:2376
```

```
$ docker context use prod-env
```

Common Mistakes / Edge Cases:

Executing critical deployment or cleanup scripts while operating in a context layer that targets production systems instead of local sandboxes.

Related Commands: `docker info`

SECTION 2: IMAGE MANAGEMENT COMMANDS

Command: `docker pull`

Purpose: Downloads an image or a repository from a registry.

Syntax:

```
docker pull [OPTIONS] NAME[:TAG|@DIGEST]
```

Options Table:

Option	Description
<code>-a, --all-tags</code>	Download all tagged images within the repository reference path match matrices.
<code>--disable-content-trust</code>	Skip image verification signatures checks during download routines entirely (Default is true).
<code>-q, --quiet</code>	Suppress descriptive operational standard logging blocks during transmission streams.

Common Usage Patterns:

- Pull an explicit microservice digest hash value directly.
- Retrieve complete tag sets.

Real-World Examples:

```
$ docker pull ubuntu:22.04
```

```
$ docker pull  
alpine@sha256:c5b1261d6d3e4385a6935406791a5059d257618522ca05d99e524e1001e4c311
```

Common Mistakes / Edge Cases:

Omitting the targeted tags inside configuration automation layers, which implicitly pulls 'latest', risking accidental structural regressions.

Related Commands: `docker images`, `docker push`

Command: `docker push`

Purpose: Uploads an image or a repository to a registry.

Syntax:

```
docker push [OPTIONS] NAME[:TAG]
```

Options Table:

Option	Description
<code>-a, --all-tags</code>	Push all tags belonging to the defined structural local registry matching path.
<code>-q, --quiet</code>	Suppress visual terminal feedback operations data streams entirely.

Common Usage Patterns:

- Upload production builds to authoritative distributions targets.

Real-World Examples:

```
$ docker push myregistry.local:5000/app:v1.4.2
```

Common Mistakes / Edge Cases:

Attempting to push custom builds without applying appropriate fully qualified multi-part remote domain tagging schemes beforehand.

Related Commands: `docker tag`, `docker login`

Command: `docker search`

Purpose: Search the Docker Hub for images.

Syntax:

```
docker search [OPTIONS] TERM
```

Options Table:

Option	Description
<code>--filter</code>	Filter output based on conditions provided (e.g., 'is-official=true', 'stars=3')
<code>--limit</code>	Maximum number of search results to return (Default: 25)

Common Usage Patterns:

- Search verified base operating systems layers.

Real-World Examples:

```
$ docker search --filter 'is-official=true' alpine
```

Common Mistakes / Edge Cases:

Relying on unverified community images which might contain severe structural security flaws.

Related Commands: `docker pull`

Command: `docker images / docker image ls`

Purpose: Lists locally stored images with details including repository tags, image IDs, creation timestamps, and virtual disk sizes.

Syntax:

```
docker images [OPTIONS] [REPOSITORY[:TAG]]
```

Options Table:

Option	Description
<code>-a, --all</code>	Show all images, including intermediate build layers that are typically hidden.
<code>--digests</code>	Show SHA256 digests explicitly in the column outputs.
<code>-f, --filter</code>	Filter images based on criteria (e.g., 'dangling=true').
<code>-q, --quiet</code>	Only show numeric IDs.

Common Usage Patterns:

- Identify orphaned layers to reclaim disk footprint properties.
- Extract image IDs for automated scripts.

Real-World Examples:

```
$ docker images
```

```
$ docker images --filter 'dangling=true'
```

```
$ docker images -q iron-app
```

Common Mistakes / Edge Cases:
Confusing dangling images (untagged build remains) with active intermediate layer structures.

Related Commands: `docker image rm`, `docker image prune`

Command: `docker image rm / docker rmi`

Purpose: Removes one or more images from the local engine storage system.

Syntax:

```
docker rmi [OPTIONS] IMAGE [IMAGE...]
```

Options Table:

Option	Description
<code>-f, --force</code>	Force removal of the image even if it is referenced by a stopped container.
<code>--no-prune</code>	Do not delete untagged parents of the removed image.

Common Usage Patterns:

- Delete a specific build image tag.
- Force removal of conflicting repository definitions.

Real-World Examples:

```
$ docker rmi alpine:latest
```

```
$ docker rmi -f e4a354e463a5
```

Common Mistakes / Edge Cases:

Attempting to remove an image without realizing a stopped container still structurally locks its underlying layer dependency tree.

Related Commands: `docker rm`, `docker image prune`

Command: `docker image inspect`

Purpose: Provides detailed low-level JSON configuration structures for an image, including layers, environment variables, entrypoints, and architecture targets.

Syntax:

```
docker image inspect [OPTIONS] IMAGE [IMAGE...]
```

Options Table:

Option	Description
<code>-f, --format</code>	Format the output using a Go template.

Common Usage Patterns:

- Extract the environment variables declared in the image layer metadata.

Real-World Examples:

```
$ docker image inspect nginx
```

```
$ docker image inspect --format='{{.Config.Env}}' nginx
```

Common Mistakes / Edge Cases:

Parsing raw text outputs instead of leveraging the `--format` template engine parameters for robust script operations.

Related Commands: `docker inspect`, `docker history`

Command: `docker image history`

Purpose: Shows the history of an image, showing the commands used to construct each layer, its size, and its creation metadata.

Syntax:

```
docker image history [OPTIONS] IMAGE
```

Options Table:

Option	Description
<code>--no-trunc</code>	Do not truncate output fields.
<code>-q, --quiet</code>	Only show numeric IDs.

Common Usage Patterns:

- Analyze layer composition for build optimization tasks.

Real-World Examples:

```
$ docker image history --no-trunc node:alpine
```

Common Mistakes / Edge Cases:

Failing to check for sensitive secrets leaked into intermediate layers during build processes.

Related Commands: `docker image inspect`

Command: `docker image save`

Purpose: Saves one or more images to a tar archive, preserving all parent layers, tags, and histories.

Syntax:

```
docker image save [OPTIONS] IMAGE [IMAGE...]
```

Options Table:

Option	Description
<code>-o, --output</code>	Write to a file instead of STDOUT.

Common Usage Patterns:

- Export an image for air-gapped system deployments.

Real-World Examples:

```
$ docker image save -o redis-latest.tar redis:latest
```

Common Mistakes / Edge Cases:

Confusing ``save`` (for images) with ``export`` (which flattens container filesystems into a single layer).

Related Commands: `docker image load`, `docker export`

Command: `docker image load`

Purpose: Loads an image from a tar archive or STDIN, restoring all contained layers, histories, and repository tags.

Syntax:

```
docker image load [OPTIONS]
```

Options Table:

Option	Description
<code>-i, --input</code>	Read from a tar archive file instead of STDIN.
<code>-q, --quiet</code>	Suppress loading progress bars reporting.

Common Usage Patterns:

- Import images onto internal nodes lacking WAN gateways.

Real-World Examples:

```
$ docker image load -i redis-latest.tar
```

Common Mistakes / Edge Cases:

Attempting to load a flattened container filesystem exported via ``docker export`` into ``docker load``, which breaks the engine metadata mappings.

Related Commands: `docker image save`, `docker import`

Command: `docker image tag`

Purpose: Creates a pointer alias referencing a source image target without duplicating physical underlying layers.

Syntax:

```
docker image tag SOURCE_IMAGE[:TAG] TARGET_IMAGE[:TAG]
```

Common Usage Patterns:

- Alias a local image to a target remote distribution path pattern.

Real-World Examples:

```
$ docker image tag local-app:latest myregistry.internal:5000/prod-app:v2.1.0
```

Common Mistakes / Edge Cases:

Typing errors in destination registry prefixes, which causes the subsequent push to target default registries.

Related Commands: `docker push`

Command: `docker image prune`

Purpose: Removes unused images to reclaim system storage space.

Syntax:

```
docker image prune [OPTIONS]
```

Options Table:

Option	Description
<code>-a, --all</code>	Remove all unused images, not just dangling ones.
<code>-f, --force</code>	Do not prompt for confirmation before deleting structures.

Common Usage Patterns:

- Perform scheduled maintenance disk clearing routines.

Real-World Examples:

```
$ docker image prune -a -f
```

Common Mistakes / Edge Cases:

Using `-a` inside dynamic automated workflows without ensuring necessary base environments are currently actively running as containers.

Related Commands: `docker system prune`

Command: `docker image import`

Purpose: Imports the contents from a tarball to create a single-layer filesystem image.

Syntax:

```
docker image import [OPTIONS] file|URL|- [REPOSITORY[:TAG]]
```

Options Table:

Option	Description
<code>-c, --change</code>	Apply Dockerfile instructions to the created image during creation routines.

Common Usage Patterns:

- Generate a minimalist base image from custom root filesystems.

Real-World Examples:

```
$ docker image import rootfs.tar custom-base:v1
```

Common Mistakes / Edge Cases:

Expecting full history metadata configurations from the source system to survive inside the imported structural object.

Related Commands: `docker export`, `docker image load`

SECTION 3: CONTAINER LIFECYCLE MANAGEMENT

Command: `docker run`

Purpose: Creates and starts a new container from a specified image, allocating filesystems, network configurations, and isolation runtimes.

Syntax:

```
docker run [OPTIONS] IMAGE [COMMAND] [ARG...]
```

Options Table:

Option	Description
<code>-d, --detached</code>	Run container in background and print container ID.
<code>-p, --publish</code>	Publish a container's port(s) to the host system (hostPort:containerPort).
<code>-v, --volume</code>	Bind mount a volume or directory path into the container environment.
<code>--name</code>	Assign a unique semantic identifier string name to the container.
<code>--restart</code>	Restart policy to apply when a container exits (no, on-failure, always, unless-stopped).
<code>-e, --env</code>	Set environment variables inside the runtime scope.
<code>--network</code>	Connect a container to a specified docker network layout layer.

Common Usage Patterns:

- Spawn interactive throwaway micro-terminals for debugging tasks.
- Run persistent background workloads with automated health recovery bounds.

Real-World Examples:

```
$ docker run -it --rm alpine sh
```

```
$ docker run -d -p 8080:80 --name web-proxy --restart unless-stopped nginx:alpine
```

```
$ docker run -e DB_HOST=10.0.0.5 -v /data:/app/data backend:v1.2
```

Common Mistakes / Edge Cases:

Forgetting that `-it` is mandatory for interactive process binaries like `bash` or `sh`, causing immediate unexpected runtime exit states.

Related Commands: docker create, docker start, docker exec

Command: docker create

Purpose: Pre-configures and writes container metadata and storage structures onto the system disk layers without initiating execution.

Syntax:

```
docker create [OPTIONS] IMAGE [COMMAND] [ARG...]
```

Options Table:

Option	Description
--name	Assign specific semantic identifier string names to the container configuration.
-v, --volume	Configure underlying storage mappings ahead of execution cycles.

Common Usage Patterns:

- Stash structural parameters during multi-step orchestration workflows.

Real-World Examples:

```
$ docker create --name service-holder -p 9000:9000 app-binary:latest
```

Common Mistakes / Edge Cases:

Expecting network configurations or internal application initializations to trigger before explicit start commands are parsed.

Related Commands: docker run, docker start

Command: `docker start`

Purpose: Starts one or more stopped or newly created containers, resuming original isolated processes.

Syntax:

```
docker start [OPTIONS] CONTAINER [CONTAINER...]
```

Options Table:

Option	Description
<code>-a, --attach</code>	Attach STDOUT/STDERR and forward signals directly to the terminal processor.
<code>-i, --interactive</code>	Attach container's STDIN to keep interactive processing streams functioning.

Common Usage Patterns:

- Wake background daemons up from suspended or configured stopped states.

Real-World Examples:

```
$ docker start web-proxy
```

```
$ docker start -ai dynamic-worker-01
```

Common Mistakes / Edge Cases:

Omitting `-i` when restarting a bash shell container, which immediately drops back into a stopped engine status phase.

Related Commands: `docker run`, `docker stop`

Command: `docker stop`

Purpose: Gracefully stops a running container by sending a SIGTERM signal, followed by a SIGKILL after a grace period timeout.

Syntax:

```
docker stop [OPTIONS] CONTAINER [CONTAINER...]
```

Options Table:

Option	Description
<code>-t, --time</code>	Seconds to wait for stop before killing it forcibly (Default: 10 seconds).

Common Usage Patterns:

- Initiate structural updates or safe application configuration shutdowns.

Real-World Examples:

```
$ docker stop database-node-01
```

```
$ docker stop -t 30 heavy-worker
```

Common Mistakes / Edge Cases:

Setting standard intervals too low for stateful engines like databases, causing structural transaction journaling corruptions.

Related Commands: `docker kill`, `docker restart`

Command: `docker restart`

Purpose: Stops and starts a container again, cycling its processes and reapplying storage attachments.

Syntax:

```
docker restart [OPTIONS] CONTAINER [CONTAINER...]
```

Options Table:

Option	Description
<code>-t, --time</code>	Seconds to wait for stop before killing the application forcibly.

Common Usage Patterns:

- Force reload changed runtime components or clear transient execution locks.

Real-World Examples:

```
$ docker restart web-server
```

Common Mistakes / Edge Cases:

Relying on restart loops to mitigate application memory leaks instead of tracking root-cause bugs.

Related Commands: `docker stop`, `docker start`

Command: `docker pause / unpause`

Purpose: Suspends or resumes all processes within the specified container using the cgroups freezer subsystem.

Syntax:

```
docker pause CONTAINER | docker unpause CONTAINER
```

Common Usage Patterns:

- Freeze processing contexts to temporarily preserve shared hardware resource capacities.

Real-World Examples:

```
$ docker pause batch-job-02
```

```
$ docker unpause batch-job-02
```

Common Mistakes / Edge Cases:

Assuming pausing releases container memory; the allocation profiles remain fully committed within kernel spaces.

Related Commands: `docker stop`

Command: `docker kill`

Purpose: Immediately sends a specified operating system signal (default: SIGKILL) to the primary process of a container.

Syntax:

```
docker kill [OPTIONS] CONTAINER [CONTAINER...]
```

Options Table:

Option	Description
<code>-s, --signal</code>	Signal to send to the container instead of default SIGKILL (e.g., SIGHUP).

Common Usage Patterns:

- Forcibly terminate non-responsive environments or issue application reloads via system signals.

Real-World Examples:

```
$ docker kill hung-container
```

```
$ docker kill -s SIGHUP nginx-router
```

Common Mistakes / Edge Cases:

Using `kill` as the standard termination pattern, which bypasses graceful application cleanup routines.

Related Commands: `docker stop`

Command: `docker wait`

Purpose: Blocks terminal execution until one or more targeted containers exit, then prints their final exit codes.

Syntax:

```
docker wait CONTAINER [CONTAINER...]
```

Common Usage Patterns:

- Sequence batch pipeline execution stages inside automation control wrappers.

Real-World Examples:

```
$ docker wait task-worker-4
```

Common Mistakes / Edge Cases:

Targeting a container that is configured with an infinite runtime loop, causing scripts to hang indefinitely.

Related Commands: `docker run`, `docker logs`

Command: `docker rm`

Purpose: Removes one or more containers from the local engine, destroying its non-persistent read-write filesystem layers.

Syntax:

```
docker rm [OPTIONS] CONTAINER [CONTAINER...]
```

Options Table:

Option	Description
<code>-f, --force</code>	Force the removal of a running container using SIGKILL structural process signals.
<code>-v, --volumes</code>	Remove anonymous volumes explicitly allocated and attached to the container target.

Common Usage Patterns:

- Purge dead environments to clean system structures clean.

Real-World Examples:

```
$ docker rm legacy-test-env
```

```
$ docker rm -fv runtime-worker-node
```

Common Mistakes / Edge Cases:

Leaving dangling anonymous volumes behind by omitting the ``-v`` flag, leading to silent disk space leaks over time.

Related Commands: `docker container prune`, `docker image rm`

Command: `docker rename`

Purpose: Modifies the structural identifier name string of an existing container without affecting its running states.

Syntax:

```
docker rename CONTAINER NEW_NAME
```

Common Usage Patterns:

- Update operational records or realign environments following version upgrades.

Real-World Examples:

```
$ docker rename staging-app-v1 staging-app-deprecated
```

Common Mistakes / Edge Cases:

Forgetting that internal link references or network aliases are not automatically adjusted by name strings modifications.

Related Commands: `docker run`

Command: `docker update`

Purpose: Dynamically updates resource limits (CPU, memory, kernel configurations) on one or more running containers.

Syntax:

```
docker update [OPTIONS] CONTAINER [CONTAINER...]
```

Options Table:

Option	Description
<code>-m, --memory</code>	Memory limit configuration boundary value specification.
<code>--cpus</code>	Number of functional CPU allocation blocks mapped into cgroups boundaries.

Common Usage Patterns:

- Throttle resource consumption during unexpected high-load incidents without causing service downtime.

Real-World Examples:

```
$ docker update --cpus 2 --memory 1024m production-api
```

Common Mistakes / Edge Cases:

Setting memory limits too low dynamically, which can trigger the kernel OOM killer to immediately terminate the containerized application.

Related Commands: `docker stats`

Command: `docker inspect`

Purpose: Returns comprehensive low-level runtime configuration layout details for containers, networks, volumes, or individual images in a structured JSON format.

Syntax:

```
docker inspect [OPTIONS] NAME|ID [NAME|ID...]
```

Options Table:

Option	Description
<code>-f, --format</code>	Format the output using a Go template engine string pattern.
<code>-s, --size</code>	Display total file size metrics if inspecting container structures.

Common Usage Patterns:

- Extract runtime IP addresses, mount source targets, or health statuses directly.

Real-World Examples:

```
$ docker inspect target-container
```

```
$ docker inspect --format='{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' target-container
```

Common Mistakes / Edge Cases:
Parsing raw JSON text arrays manually in custom shell wrapper frameworks instead of using built-in Go formatting paths.

Related Commands: `docker image inspect`

Command: `docker top`

Purpose: Displays the active processes running inside a container, mapping container PIDs to their corresponding host system PIDs.

Syntax:

```
docker top CONTAINER [ps OPTIONS]
```

Common Usage Patterns:

- Audit internal container processes from the security context of the host operating system layer.

Real-World Examples:

```
$ docker top security-vault-app
```

```
$ docker top security-vault-app -ef
```

Common Mistakes / Edge Cases:

Assuming that container process engines are completely separate from host kernels; they are isolated but run directly on the host kernel scheduler.

Related Commands: `docker stats`, `docker exec`

Command: `docker stats`

Purpose: Displays a live, real-time streaming dashboard of container resource usage statistics, including CPU, memory, network I/O, and block I/O.

Syntax:

```
docker stats [OPTIONS] [CONTAINER...]
```

Options Table:

Option	Description
<code>-a, --all</code>	Show metrics for all containers, including stopped ones.
<code>--no-stream</code>	Disable the live streaming interface and print a single snapshot of current usage metrics.

Common Usage Patterns:

- Monitor cluster health status indicators during load testing phases.

Real-World Examples:

```
$ docker stats --no-stream API-gateway-01
```

Common Mistakes / Edge Cases:

Running unthrottled streaming queries over high-latency remote SSH daemon setups, consuming unnecessary network resources.

Related Commands: `docker top`

Command: `docker logs`

Purpose: Fetches and streams historical or current execution logs generated by processes running within a container via STDOUT and STDERR channels.

Syntax:

```
docker logs [OPTIONS] CONTAINER
```

Options Table:

Option	Description
<code>-f, --follow</code>	Follow the live log output stream continuously.
<code>--since</code>	Show logs generated since a specific timestamp or duration (e.g., '2h' or ISO 8601 format).
<code>-t, --timestamps</code>	Prepend clear human-readable timestamps to every log line entry.
<code>--tail</code>	Number of log lines to show from the end of the history buffer (Default: 'all').

Common Usage Patterns:

- Diagnose structural runtime crashes or monitor application log output streams.

Real-World Examples:

```
$ docker logs -f --tail 100 app-backend
```

```
$ docker logs --since 10m microservice-auth
```

Common Mistakes / Edge Cases:

Attempting to pull logs from containers configured with external logging drivers like Syslog or Fluentd, which disables local ``docker logs`` caching pipelines.

Related Commands: `docker events`

Command: `docker attach`

Purpose: Attaches local standard input, output, and error streams directly to a running container's primary execution process (PID 1).

Syntax:

```
docker attach [OPTIONS] CONTAINER
```

Options Table:

Option	Description
<code>--no-stdin</code>	Do not attach the STDIN stream channel properties.

Common Usage Patterns:

- Interact directly with the main running application process loop.

Real-World Examples:

```
$ docker attach interactive-console-service
```

Common Mistakes / Edge Cases:

Pressing Ctrl+C in an attached terminal view without realizing it sends a SIGINT signal that terminates the container's primary PID 1 process.

Related Commands: `docker exec`

Command: `docker exec`

Purpose: Spawns a new independent execution process inside the namespaces of an already running container.

Syntax:

```
docker exec [OPTIONS] CONTAINER COMMAND [ARG...]
```

Options Table:

Option	Description
<code>-d, --detach</code>	Run the command in the background (detached mode).
<code>-i, --interactive</code>	Keep STDIN open even if not attached.
<code>-t, --tty</code>	Allocate a pseudo-TTY terminal window construct.
<code>-u, --user</code>	Specify the username or UID to execute the process under.

Common Usage Patterns:

- Open a debugging shell inside a live production container, or run remote administrative utility commands.

Real-World Examples:

```
$ docker exec -it web-app bash
```

```
$ docker exec -u root stateful-db mysql
```

Common Mistakes / Edge Cases:

Attempting to run commands inside a stopped container, which is unsupported since namespaces require an active process context.

Related Commands: `docker run`, `docker attach`

Command: `docker cp`

Purpose: Copies files or directories between a container's filesystem and the host machine's local filesystem paths.

Syntax:

```
docker cp [OPTIONS] CONTAINER:SRC_PATH DEST_PATH | SRC_PATH CONTAINER:DEST_PATH
```

Options Table:

Option	Description
<code>-a, --archive</code>	Preserve original UID/GID properties and metadata timestamps during transfer processing.

Common Usage Patterns:

- Extract diagnostic core dumps, or inject emergency configuration updates directly into file systems.

Real-World Examples:

```
$ docker cp core-app:/var/log/app.dump ./diagnostics/
```

```
$ docker cp ./patched.conf core-app:/etc/app/app.conf
```

Common Mistakes / Edge Cases:

Assuming target directories will be created recursively if they do not exist, which can cause file placement errors.

Related Commands: `docker volume`

Command: `docker diff`

Purpose: Inspects and reports changes made to a container's read-write layer filesystem compared to its originating image structure.

Syntax:

```
docker diff CONTAINER
```

Common Usage Patterns:

- A: Added files, C: Changed files, D: Deleted files classification logs auditing.

Real-World Examples:

```
$ docker diff stateful-worker
```

Common Mistakes / Edge Cases:

Failing to notice untracked file generation loops that inflate container disk usage outside explicit volume areas.

Related Commands: `docker commit`

Command: `docker port`

Purpose: Lists the host public port mappings that forward traffic to the container's private isolated network ports.

Syntax:

```
docker port CONTAINER [PRIVATE_PORT[/PROTO]]
```

Common Usage Patterns:

- Verify routing paths when troubleshooting multi-tier container communications.

Real-World Examples:

```
$ docker port load-balancer
```

```
$ docker port load-balancer 80/tcp
```

Common Mistakes / Edge Cases:

Assuming ports are globally exposed without checking if they are bound specifically to localhost (127.0.0.1) or public interfaces (0.0.0.0).

Related Commands: `docker run`, `docker network`

Command: `docker commit`

Purpose: Creates a new image snapshot from a container's current read-write filesystem layer modifications.

Syntax:

```
docker commit [OPTIONS] CONTAINER [REPOSITORY[:TAG]]
```

Options Table:

Option	Description
<code>-a, --author</code>	Author string information details configuration payload metadata.
<code>-c, --change</code>	Apply Dockerfile instructions (e.g., CMD, ENV) to the newly created image layer layout.
<code>-m, --message</code>	Commit message describing the modifications performed inside the container structures.

Common Usage Patterns:

- Capture state snapshots of experimental environments for later debugging analysis.

Real-World Examples:

```
$ docker commit -m 'installed debugging tools' experimental-box debug-image:v1
```

Common Mistakes / Edge Cases:

Using `commit` as a standard substitute for Dockerfile-based declarative image building, which compromises repeatability and transparency.

Related Commands: `docker image save`, `docker build`

Command: `docker export`

Purpose: Exports a container's entire active filesystem as a flat tar archive file, discarding all history metadata and underlying layer definitions.

Syntax:

```
docker export [OPTIONS] CONTAINER
```

Options Table:

Option	Description
<code>-o, --output</code>	Write to a file instead of standard output stream protocols.

Common Usage Patterns:

- Flatten container structures into single-layer distributions frameworks.

Real-World Examples:

```
$ docker export -o flattened-rootfs.tar app-instance-01
```

Common Mistakes / Edge Cases:

Expecting environment variables or entrypoints declared in the original image to persist when importing a flattened tarball.

Related Commands: `docker image import`, `docker image save`

SECTION 4: BUILD COMMANDS

Command: `docker build`

Purpose: Builds a Docker image from a declarative Dockerfile and a specified build context path or URL.

Syntax:

```
docker build [OPTIONS] PATH | URL | -
```

Options Table:

Option	Description
<code>-t, --tag</code>	Name and optionally a tag in the 'name:tag' format definition pattern parameters.
<code>-f, --file</code>	Name of the Dockerfile to parse explicitly (Default is 'Dockerfile' in context root).
<code>--no-cache</code>	Do not use cached layers during the image build process.
<code>--build-arg</code>	Set build-time variables that can be accessed inside the Dockerfile compilation phase.

Common Usage Patterns:

- Compile container images locally using structural optimization boundaries.

Real-World Examples:

```
$ docker build -t system-service:v1.0 .
```

```
$ docker build -f /path/to/custom.Dockerfile -t customized-app --no-cache .
```

Common Mistakes / Edge Cases:
Passing large, unindexed directories containing build artifacts as the context path, which delays the build while uploading data to the daemon.

Related Commands: `docker builder`, `docker buildx`

Command: `docker builder`

Purpose: Provides subcommands to manage BuildKit build instances, disk caches, and build engines settings properties.

Syntax:

```
docker builder COMMAND
```

Options Table:

Option	Description
<code>prune</code>	Clean up the BuildKit build cache entirely to reclaim system disk space.
<code>du</code>	Disk usage analysis reporting for internal build engine assets buffers.

Common Usage Patterns:

- Reclaim cached space left over from complex multi-stage image builds.

Real-World Examples:

```
$ docker builder du
```

```
$ docker builder prune -f
```

Common Mistakes / Edge Cases:

Forgetting that clearing BuildKit caches forces subsequent builds to re-download dependencies and rebuild layers from scratch.

Related Commands: `docker build`, `docker system prune`

Command: `docker buildx`

Purpose: Extends standard build capabilities by leveraging BuildKit to enable multi-architecture cross-compilation, concurrent building, and advanced cache management.

Syntax:

```
docker buildx COMMAND
```

Options Table:

Option	Description
<code>create</code>	Create a new builder instance configuration node targeting cross-compilation workloads.
<code>ls</code>	List all active builder instances and their supported target architectures (ARM, AMD64).
<code>build</code>	Execute a build utilizing BuildKit's advanced multi-architecture target definitions.

Common Usage Patterns:

- Build unified multi-platform container images for diverse compute hardware architectures.

Real-World Examples:

```
$ docker buildx ls
```

```
$ docker buildx create --name multi-platform-builder --use
```

Common Mistakes / Edge Cases:

Attempting to push multi-architecture builds without configuring a remote registry target, since multi-arch manifests cannot be stored in the legacy local container cache layout.

Related Commands: `docker build`, `docker context`

SECTION 5: VOLUME ENGINE COMMANDS

Command: `docker volume create`

Purpose: Creates a persistent storage volume managed by the Docker daemon, independent of any single container lifecycle.

Syntax:

```
docker volume create [OPTIONS] [VOLUME]
```

Options Table:

Option	Description
<code>-d, --driver</code>	Specify the volume driver type to use (Default is 'local').
<code>--label</code>	Set metadata labels on the volume for organization or filtering.
<code>-o, --opt</code>	Set driver-specific options (e.g., configuring NFS mount endpoints).

Common Usage Patterns:

- Provision dedicated persistent storage spaces for database state preservation layers.

Real-World Examples:

```
$ docker volume create database-storage
```

Common Mistakes / Edge Cases:
Failing to pass driver-specific parameters correctly, which can result in silent runtime errors when containers try to mount the storage.

Related Commands: `docker volume ls`, `docker run`

Command: `docker volume ls`

Purpose: Lists all persistent volumes managed by the local Docker engine.

Syntax:

```
docker volume ls [OPTIONS]
```

Options Table:

Option	Description
<code>-f, --filter</code>	Filter the volume list based on criteria (e.g., 'dangling=true').
<code>-q, --quiet</code>	Only display the volume names.

Common Usage Patterns:

- Identify detached or orphaned volumes consuming disk space.

Real-World Examples:

```
$ docker volume ls
```

```
$ docker volume ls --filter dangling=true
```

Common Mistakes / Edge Cases:

Confusing anonymous volumes with named volumes, leading to the accidental preservation of stale temp data.

Related Commands: `docker volume prune`

Command: `docker volume inspect`

Purpose: Displays detailed low-level JSON configuration details for a specific volume, including its mount path on the host filesystem.

Syntax:

```
docker volume inspect [OPTIONS] VOLUME [VOLUME...]
```

Common Usage Patterns:

- Determine the exact host directory backing a managed volume for backup purposes.

Real-World Examples:

```
$ docker volume inspect database-storage
```

Common Mistakes / Edge Cases:

Modifying files directly in the host mount path returned by inspect, which can corrupt the volume's metadata and access controls.

Related Commands: `docker inspect`

Command: `docker volume rm`

Purpose: Removes one or more specified volumes from the engine storage layer.

Syntax:

```
docker volume rm [OPTIONS] VOLUME [VOLUME...]
```

Options Table:

Option	Description
<code>-f, --force</code>	Force the removal of a volume (bypassing minor active validation checks).

Common Usage Patterns:

- Purge unneeded storage spaces during application decommissioning.

Real-World Examples:

```
$ docker volume rm legacy-data
```

Common Mistakes / Edge Cases:

Attempting to delete a volume that is still referenced by a stopped container, which results in a deletion error.

Related Commands: `docker rm`, `docker volume prune`

Command: `docker volume prune`

Purpose: Deletes all unreferenced volumes, clearing unused persistent storage spaces.

Syntax:

```
docker volume prune [OPTIONS]
```

Options Table:

Option	Description
<code>-f, --force</code>	Do not prompt for confirmation before deleting structures.

Common Usage Patterns:

- Perform automated system disk space reclamation routines.

Real-World Examples:

```
$ docker volume prune -f
```

Common Mistakes / Edge Cases:

Running prune in environments where stateful containers are temporarily uncreated, risking irreversible loss of application data.

Related Commands: `docker system prune`, `docker volume rm`

SECTION 6: NETWORK ARCHITECTURE COMMANDS

Command: `docker network create`

Purpose: Creates a new network infrastructure topology layer that enables isolated communication between connected containers.

Syntax:

```
docker network create [OPTIONS] NETWORK
```

Options Table:

Option	Description
<code>-d, --driver</code>	Driver to manage the network topology (bridge, overlay, macvlan, host, none).
<code>--subnet</code>	Specify the network subnet in CIDR format (e.g., 172.20.0.0/16).
<code>--gateway</code>	IPv4 or IPv6 gateway configuration parameter for the master subnet mask.
<code>--internal</code>	Restrict external public internet access into or out of this network block boundary.

Common Usage Patterns:

- Establish isolated multi-tier networks for secure container-to-container routing.

Real-World Examples:

```
$ docker network create --driver bridge isolated-backend
```

```
$ docker network create --subnet=192.168.100.0/24 --gateway=192.168.100.1 private-net
```

Common Mistakes / Edge Cases:
Overlapping custom network subnets with existing physical network ranges on the host system, which causes routing conflicts.

Related Commands: `docker network ls`, `docker network connect`

Command: `docker network connect`

Purpose: Connects a running container to an existing network interface layer dynamically.

Syntax:

```
docker network connect [OPTIONS] NETWORK CONTAINER
```

Options Table:

Option	Description
<code>--ip</code>	Allocate a specific static IPv4 address to the container interface.
<code>--alias</code>	Add a network-scoped domain name alias for DNS resolution within that network.

Common Usage Patterns:

- Bridge an existing container into an additional network tier for cross-service integration.

Real-World Examples:

```
$ docker network connect isolated-backend web-proxy
```

```
$ docker network connect --ip 172.20.0.55 secure-net api-server
```

Common Mistakes / Edge Cases:

Forgetting that a container can connect to multiple networks simultaneously, and assigning conflicting network aliases.

Related Commands: `docker network disconnect`

Command: `docker network disconnect`

Purpose: Disconnects a container from a specified network interface layer dynamically.

Syntax:

```
docker network disconnect [OPTIONS] NETWORK CONTAINER
```

Options Table:

Option	Description
<code>-f, --force</code>	Force the container to disconnect from the target network.

Common Usage Patterns:

- Isolate compromised or deprecated container instances from internal communication channels.

Real-World Examples:

```
$ docker network disconnect isolated-backend legacy-worker
```

Common Mistakes / Edge Cases:

Assuming disconnection terminates the container; it remains running but loses its network interface for that segment.

Related Commands: `docker network connect`

Command: `docker network inspect`

Purpose: Displays detailed low-level JSON configuration details for a network, including all connected containers and their allocated IP addresses.

Syntax:

```
docker network inspect [OPTIONS] NETWORK [NETWORK...]
```

Common Usage Patterns:

- Map out current container network placements and audit internal IP allocations.

Real-World Examples:

```
$ docker network inspect isolated-backend
```

Common Mistakes / Edge Cases:

Parsing raw inspection outputs during high-velocity container scaling phases, where IP states may change dynamically.

Related Commands: `docker inspect`

Command: `docker network ls`

Purpose: Lists all networks available on the local Docker engine or Swarm cluster manager.

Syntax:

```
docker network ls [OPTIONS]
```

Options Table:

Option	Description
<code>-f, --filter</code>	Filter networks based on specific conditions (e.g., 'driver=bridge').
<code>-q, --quiet</code>	Only display numeric network IDs.

Common Usage Patterns:

- Audit available network drivers and drivers layouts across the host system.

Real-World Examples:

```
$ docker network ls
```

```
$ docker network ls --filter driver=overlay
```

Common Mistakes / Edge Cases:
Attempting to delete built-in networks (bridge, host, none) which are protected core assets of the Docker engine topology.

Related Commands: `docker network rm`

Command: `docker network rm`

Purpose: Removes one or more user-defined networks from the configuration layers.

Syntax:

```
docker network rm NETWORK [NETWORK...]
```

Common Usage Patterns:

- Purge obsolete network configurations during infrastructure teardowns.

Real-World Examples:

```
$ docker network rm private-net
```

Common Mistakes / Edge Cases:

Attempting to remove a network that still has active container endpoints attached, which triggers a resource busy error.

Related Commands: `docker network prune`

Command: `docker network prune`

Purpose: Removes all unreferenced, user-defined networks to clean up the network subsystem.

Syntax:

```
docker network prune [OPTIONS]
```

Options Table:

Option	Description
<code>-f, --force</code>	Do not prompt for confirmation before deleting structures.

Common Usage Patterns:

- Automate the removal of unused network segments in CI/CD pipeline environments.

Real-World Examples:

```
$ docker network prune -f
```

Common Mistakes / Edge Cases:

Running network prune when dynamic container creation scripts are paused, causing dependent network components to be cleared.

Related Commands: `docker system prune`, `docker network rm`

SECTION 7: SYSTEM LEVEL ENGINE COMMANDS

Command: `docker system df`

Purpose: Displays information regarding the real-time disk space usage of the Docker daemon, broken down by images, containers, local volumes, and build caches.

Syntax:

```
docker system df [OPTIONS]
```

Options Table:

Option	Description
<code>-v, --verbose</code>	Show detailed, itemized breakdown listings of disk utilization per resource asset.

Common Usage Patterns:

- Perform high-level disk capacity planning and identify resource consumption anomalies.

Real-World Examples:

```
$ docker system df
```

```
$ docker system df -v
```

Common Mistakes / Edge Cases:
Misinterpreting reclaimable percentages and accidentally deleting important volumes during emergency cleanups.

Related Commands: `docker info`, `docker system prune`

Command: `docker system events`

Purpose: Streams real-time event logs directly from the Docker daemon engine, capturing state changes for containers, images, plugins, networks, and volumes.

Syntax:

```
docker system events [OPTIONS]
```

Options Table:

Option	Description
<code>--since</code>	Show historical engine events recorded since a specific timestamp baseline target.
<code>--until</code>	Stream events until a specific target boundary window arrives.
<code>-f, --filter</code>	Filter the real-time event stream based on criteria (e.g., 'container=web-app').

Common Usage Patterns:

- Monitor cluster operations dynamically or pipe daemon activity logs into external auditing frameworks.

Real-World Examples:

```
$ docker system events
```

```
$ docker system events --filter 'event=die'
```

Common Mistakes / Edge Cases:

Failing to specify filters when streaming events on busy production nodes, resulting in log message flooding.

Related Commands: `docker logs`

Command: `docker system info`

Purpose: An alias referencing the comprehensive core system info architecture mapping functions.

Syntax:

```
docker system info
```

Common Usage Patterns:

- Verify node capabilities during infrastructure inventory tracking routines.

Real-World Examples:

```
$ docker system info
```

Common Mistakes / Edge Cases:

Confusing system info output structures with low-level kernel tracing telemetry logs.

Related Commands: `docker version`

Command: `docker system prune`

Purpose: An aggressive cleanup utility that deletes all unused containers, networks, dangling images, and optionally, unused volumes.

Syntax:

```
docker system prune [OPTIONS]
```

Options Table:

Option	Description
<code>-a, --all</code>	Remove all unused images, not just dangling ones, along with stopped containers.
<code>--volumes</code>	Include unreferenced volumes in the aggressive cleanup pass (High risk of data loss).
<code>-f, --force</code>	Bypass interactive confirmation dialog prompts completely.

Common Usage Patterns:

- Reset staging environments or perform emergency space reclamation on full disks.

Real-World Examples:

```
$ docker system prune
```

```
$ docker system prune -a --volumes -f
```

Common Mistakes / Edge Cases:

Running system prune with the `--volumes` flag without checking for stopped stateful containers, leading to irreversible production data loss.

Related Commands: `docker image prune`, `docker volume prune`

SECTION 8: DIAGNOSTICS, LOGGING & TROUBLESHOOTING

Command: `docker logs (Diagnostic Context)`

Purpose: Used to capture error traces from failing containers by extracting standard stream histories.

Syntax:

```
docker logs --timestamps --tail 50 CONTAINER
```

Common Usage Patterns:

- Debug startup application failure loops and extract error stack traces.

Real-World Examples:

```
$ docker logs --timestamps --tail 50 crashed-app
```

Common Mistakes / Edge Cases:

Forgetting that logs are cleared when a container is removed with ``docker rm``, losing diagnostic data.

Related Commands: `docker inspect`

Command: `docker inspect (Diagnostic Context)`

Purpose: Extracts low-level state indicators, exit codes, and healthcheck failure logs from container metadata configurations.

Syntax:

```
docker inspect --format='{{json .State}}' CONTAINER
```

Common Usage Patterns:

- Query detailed structural exit states to diagnose OOM killings or application configuration issues.

Real-World Examples:

```
$ docker inspect --format='{{json .State}}' API-gateway
```

Common Mistakes / Edge Cases:

Overlooking the ``OOMKilled`` boolean indicator field inside the structural state JSON payload during debugging.

Related Commands: `docker system events`

SECTION 9: DOCKER COMPOSE COMMANDS

REFERENCE

Command: `docker compose up`

Purpose: Parses the compose YAML manifest file to build, create, start, and attach to containers for a multi-container application stack.

Syntax:

```
docker compose up [OPTIONS] [SERVICE...]
```

Options Table:

Option	Description
<code>-d, --detach</code>	Run containers in the background and print new execution parameters.
<code>--build</code>	Force re-compilation of custom images before initializing the container stack structures.
<code>--remove-orphans</code>	Remove containers for services not defined in the current compose specification file.

Common Usage Patterns:

- Initialize localized multi-tier app environments with a single command string.

Real-World Examples:

```
$ docker compose up -d
```

```
$ docker compose up --build --remove-orphans
```

Common Mistakes / Edge Cases:

Modifying application code and expecting updates to apply without passing the `--build` option, running stale image layers instead.

Related Commands: `docker compose down`

Command: `docker compose down`

Purpose: Stops and removes containers, networks, volumes, and images created by ``docker compose up``.

Syntax:

```
docker compose down [OPTIONS]
```

Options Table:

Option	Description
<code>-v, --volumes</code>	Remove named and anonymous volumes declared in the compose configuration layout file.
<code>--rmi</code>	Remove images used by services (type: 'all' or 'local').

Common Usage Patterns:

- Tear down multi-service stack environments cleanly to free system resources.

Real-World Examples:

```
$ docker compose down -v
```

```
$ docker compose down --rmi local
```

Common Mistakes / Edge Cases:
Omitting the ``-v`` flag when attempting a clean database state initialization reset, leaving stale data inside persistent volumes.

Related Commands: `docker compose up`

Command: `docker compose build`

Purpose: Builds or rebuilds services defined with a ``build:`` block in the project's compose YAML manifest.

Syntax:

```
docker compose build [OPTIONS] [SERVICE...]
```

Options Table:

Option	Description
<code>--no-cache</code>	Do not use cache layers during the multi-service build execution pass.
<code>--pull</code>	Always attempt to pull a newer version of the parent base image layer targets.

Common Usage Patterns:

- Pre-compile localized image configurations across multi-service architecture layouts.

Real-World Examples:

```
$ docker compose build --no-cache backend-worker
```

Common Mistakes / Edge Cases:

Forgetting that ``compose build`` only updates images; you must run ``compose up`` afterward to cycle running containers to use the new builds.

Related Commands: `docker build`

Command: `docker compose pull / push`

Purpose: Pulls or pushes image assets associated with services defined within the configuration stack layouts.

Syntax:

```
docker compose pull [OPTIONS] | docker compose push [OPTIONS]
```

Common Usage Patterns:

- Distribute or sync localized stack build parameters across target nodes.

Real-World Examples:

```
$ docker compose pull
```

```
$ docker compose push
```

Common Mistakes / Edge Cases:

Omitting registry tag credentials parameters inside environment settings before executing transfer actions.

Related Commands: `docker pull`

Command: `docker compose start / stop / restart`

Purpose: Controls execution states for existing container objects managed within the compose project structure without modifying definitions.

Syntax:

```
docker compose [start|stop|restart] [SERVICE...]
```

Common Usage Patterns:

- Cycle service availability fields or toggle environments during maintenance windows.

Real-World Examples:

```
$ docker compose stop db
```

```
$ docker compose restart cache-node
```

Common Mistakes / Edge Cases:

Confusing ``compose stop`` with ``compose down``, which keeps network and container layouts allocation bindings on disk space components.

Related Commands: `docker stop`

Command: `docker compose logs`

Purpose: Aggregates and displays interleaved log outputs from all active services within the current compose stack context.

Syntax:

```
docker compose logs [OPTIONS] [SERVICE...]
```

Options Table:

Option	Description
<code>-f, --follow</code>	Stream live log updates from all active services concurrently.
<code>--tail</code>	Number of lines to show from the end of the log buffer for each service container.

Common Usage Patterns:

- Debug runtime inter-service communication issues across the application stack layers.

Real-World Examples:

```
$ docker compose logs -f --tail 20 web-gateway
```

Common Mistakes / Edge Cases:

Failing to filter log outputs by passing specific service names in large stacks, resulting in unreadable terminal output.

Related Commands: `docker logs`

Command: `docker compose exec`

Purpose: Executes arbitrary commands inside the running container instance of a managed service definition.

Syntax:

```
docker compose exec [OPTIONS] SERVICE COMMAND [ARG...]
```

Options Table:

Option	Description
<code>-u, --user</code>	Specify the username or UID to execute the process under inside the target container context.

Common Usage Patterns:

- Open a database terminal console interface or inspect operational file states within service boundaries.

Real-World Examples:

```
$ docker compose exec db psql -U postgres
```

Common Mistakes / Edge Cases:

Targeting services that have scaled to multiple instances without realizing that `exec` defaults to the first instance container by default.

Related Commands: `docker exec`

Command: `docker compose run`

Purpose: Spawns a new, one-off container instance to execute an isolated utility command associated with a defined service.

Syntax:

```
docker compose run [OPTIONS] SERVICE [COMMAND] [ARG...]
```

Options Table:

Option	Description
<code>--rm</code>	Automatically remove the one-off container structure upon execution process completion.
<code>-T</code>	Disable pseudo-TTY allocation to allow piping raw data streams smoothly.

Common Usage Patterns:

- Run database migrations or schema upgrades using service environment contexts safely.

Real-World Examples:

```
$ docker compose run --rm backend-api npm run db:migrate
```

Common Mistakes / Edge Cases:

Forgetting that `compose run` spawns a *new* container instance rather than reusing existing running services, which can cause network port mapping duplication errors.

Related Commands: `docker compose exec`

Command: `docker compose ps`

Purpose: Lists the current runtime states, port mappings, and health indicators for all containers bound to the local compose project.

Syntax:

```
docker compose ps [OPTIONS]
```

Options Table:

Option	Description
<code>-a, --all</code>	Show all containers, including those that have exited or stopped.

Common Usage Patterns:

- Verify stack status parameters and inspect port mappings quickly.

Real-World Examples:

```
$ docker compose ps -a
```

Common Mistakes / Edge Cases:
Assuming the output reflects all containers on the host, rather than only those tracked by the local compose file context.

Related Commands: `docker ps`

Command: `docker compose config`

Purpose: Parses, validates, and prints the consolidated configuration model compiled from active compose files and environment variable states.

Syntax:

```
docker compose config [OPTIONS]
```

Options Table:

Option	Description
<code>-q, --quiet</code>	Validate configuration syntax blocks and exit silently without printing the model.

Common Usage Patterns:

- Validate syntax soundness across complex nested or extended compose manifest structures before deploying changes.

Real-World Examples:

```
$ docker compose config
```

Common Mistakes / Edge Cases:

Deploying stacks with unexpanded environment variable mappings due to skipping syntax validation checks.

Related Commands: `docker compose up`

Command: `docker compose images`

Purpose: Lists all images utilized by the services configured within the active compose stack layout definition.

Syntax:

```
docker compose images [OPTIONS]
```

Common Usage Patterns:

- Audit image versions and digest references across all deployed stack layers.

Real-World Examples:

```
$ docker compose images
```

Common Mistakes / Edge Cases:

Assuming the listed images are automatically updated to match remote registries without executing a pull command first.

Related Commands: `docker images`

SECTION 10: SWARM ORCHESTRATION ENGINE

Command: `docker swarm init`

Purpose: Initializes a local Docker Swarm manager node, establishing consensus engines and cryptographic root authorities.

Syntax:

```
docker swarm init [OPTIONS]
```

Options Table:

Option	Description
<code>--advertise-addr</code>	Specify the explicit network interface IP address that other swarm nodes use to connect to this manager.
<code>--listen-addr</code>	Listen address for internal swarm coordination traffic (Default: 0.0.0.0:2377).

Common Usage Patterns:

- Bootstrap high-availability orchestration clusters on native system infrastructures.

Real-World Examples:

```
$ docker swarm init --advertise-addr 10.0.0.10
```

Common Mistakes / Edge Cases:
Omitting the advertise address on systems with multiple network interfaces, which can lead to nodes joining via unreachable interfaces.

Related Commands: `docker swarm join`

Command: `docker swarm join`

Purpose: Connects a worker or manager node to an active Swarm cluster using authentication tokens.

Syntax:

```
docker swarm join [OPTIONS] HOST:PORT
```

Options Table:

Option	Description
<code>--token</code>	The authentication token required to gain cluster ingress authorization mappings.

Common Usage Patterns:

- Scale compute capacity horizontally by registering new infrastructure nodes into the cluster.

Real-World Examples:

```
$ docker swarm join --token SWMTKN-1-39gq 10.0.0.10:2377
```

Common Mistakes / Edge Cases:

Leaking joining tokens into public configuration code bases, allowing unauthenticated node access.

Related Commands: `docker swarm init`, `docker node ls`

Command: `docker swarm leave`

Purpose: Disconnects the current node from the Swarm cluster, removing its cluster membership metadata.

Syntax:

```
docker swarm leave [OPTIONS]
```

Options Table:

Option	Description
<code>-f, --force</code>	Force the node to leave the swarm, bypassing coordination protocols (Required for the last manager node).

Common Usage Patterns:

- Decommission server hardware or isolate failing compute instances safely.

Real-World Examples:

```
$ docker swarm leave --force
```

Common Mistakes / Edge Cases:

Forcing a manager node offline without draining workload tasks first, causing abrupt service disruptions.

Related Commands: `docker node rm`

Command: `docker service create`

Purpose: Creates a replicated or global service across the Swarm cluster compute infrastructure nodes.

Syntax:

```
docker service create [OPTIONS] IMAGE [COMMAND] [ARG...]
```

Options Table:

Option	Description
<code>--replicas</code>	Number of concurrent, isolated task containers to distribute across the node cluster.
<code>--name</code>	Semantic identifier string name for the service layer abstraction.
<code>--publish</code>	Publish a service port cluster-wide using the routing mesh infrastructure.

Common Usage Patterns:

- Deploy resilient, auto-load-balanced applications across distributed hardware architectures.

Real-World Examples:

```
$ docker service create --name scalable-web --replicas 5 -p 80:80 nginx:alpine
```

Common Mistakes / Edge Cases:

Using specific host-bound mount paths in non-shared storage environments, causing tasks to fail on nodes lacking those exact file configurations.

Related Commands: `docker service update`, `docker stack deploy`

Command: `docker service update`

Purpose: Dynamically updates running service parameters (image versions, environment settings, resources) across the cluster with zero downtime.

Syntax:

```
docker service update [OPTIONS] SERVICE
```

Options Table:

Option	Description
<code>--image</code>	Specify a new container image target to initiate a rolling update pattern across tasks.
<code>--replicas</code>	Scale service task counts up or down across active compute infrastructure nodes.

Common Usage Patterns:

- Perform rolling rollouts of new application versions across production clusters safely.

Real-World Examples:

```
$ docker service update --image app-backend:v2.0.0 backend-service
```

Common Mistakes / Edge Cases:
Applying sweeping configuration changes without setting appropriate parallelism bounds, which risks taking all tasks offline simultaneously.

Related Commands: `docker service rm`

Command: `docker service rm`

Purpose: Removes a service from the Swarm cluster, immediately terminating all associated tasks and container instances.

Syntax:

```
docker service rm SERVICE [SERVICE...]
```

Common Usage Patterns:

- Decommission application stacks across cluster infrastructure layers.

Real-World Examples:

```
$ docker service rm scalable-web
```

Common Mistakes / Edge Cases:

Executing service removal without backups, which permanently destroys all active ephemeral runtime filesystems.

Related Commands: `docker stack rm`

Command: `docker node ls`

Purpose: Lists all nodes registered in the Swarm cluster along with their operational roles, health statuses, and availability states.

Syntax:

```
docker node ls [OPTIONS]
```

Common Usage Patterns:

- Audit cluster hardware states and identify offline or degraded cluster nodes.

Real-World Examples:

```
$ docker node ls
```

Common Mistakes / Edge Cases:

Attempting to execute node queries on worker nodes; this metadata is restricted to active Swarm manager nodes.

Related Commands: `docker swarm init`

Command: `docker stack deploy`

Purpose: Deploys or updates a multi-service multi-node application stack to the Swarm cluster using a Compose file format definition.

Syntax:

```
docker stack deploy [OPTIONS] STACK
```

Options Table:

Option	Description
<code>-c, --compose-file</code>	Path to the Compose configuration manifest specifying the multi-service layout.

Common Usage Patterns:

- Orchestrate enterprise-grade multi-service architectures across distributed production clusters cleanly.

Real-World Examples:

```
$ docker stack deploy -c production-stack.yml enterprise-app
```

Common Mistakes / Edge Cases:

Omitting the Swarm-specific deploy block configurations within the file, causing scale properties to be ignored.

Related Commands: `docker stack rm`, `docker service create`

Command: `docker stack rm`

Purpose: Removes an entire multi-service stack from the Swarm cluster, deleting all associated services, networks, and tasks.

Syntax:

```
docker stack rm STACK [STACK...]
```

Common Usage Patterns:

- Clean up complete multi-tier application environments from the cluster infrastructure with one command.

Real-World Examples:

```
$ docker stack rm enterprise-app
```

Common Mistakes / Edge Cases:

Forgetting that `stack rm` does not clean up persistent data volumes, which must be managed separately on individual host nodes.

Related Commands: `docker stack deploy`

SECTION 11: REGISTRY ENGINE COMMANDS

Command: `docker login`

Purpose: Authenticates the local CLI client against a remote image registry, saving credentials securely in system store files.

Syntax:

```
docker login [OPTIONS] [SERVER]
```

Options Table:

Option	Description
<code>-u, --username</code>	Registry authentication account username identification parameter string value.
<code>-p, --password</code>	Registry authentication security credential access token string password pass.

Common Usage Patterns:

- Authenticate sessions targeting commercial registries or private image distribution repositories securely.

Real-World Examples:

```
$ docker login myregistry.internal:5000 -u admin-user -p token
```

Common Mistakes / Edge Cases:
Passing raw passwords via CLI flags in shared execution bash history frames, leaking credentials to shell histories.

Related Commands: `docker logout`

Command: `docker logout`

Purpose: Terminates active sessions and clears authentication credentials for a specified remote registry from local storage.

Syntax:

```
docker logout [SERVER]
```

Common Usage Patterns:

- De-authorize build node tokens following pipeline finalization loops.

Real-World Examples:

```
$ docker logout myregistry.internal:5000
```

Common Mistakes / Edge Cases:

Leaving long-lived session tokens cached on shared development machines, risking unauthorized image modifications.

Related Commands: `docker login`

SECTION 12: AGGRESSIVE DISK & RESOURCE CLEANUP

Command: `docker container prune`

Purpose: Deletes all stopped containers from the host machine to reclaim disk space.

Syntax:

```
docker container prune [OPTIONS]
```

Options Table:

Option	Description
<code>-f, --force</code>	Bypass confirmation dialogs completely.

Common Usage Patterns:

- Clean up accumulated ad-hoc testing containers from dev environments.

Real-World Examples:

```
$ docker container prune -f
```

Common Mistakes / Edge Cases:

Running prune while debugging complex, multi-container interactions and accidentally deleting needed containers that were temporarily stopped.

Related Commands: `docker rm`, `docker system prune`

APPENDIX A: DOCKER COMMAND CHEAT SHEET

Command Matrix	Target Intent Workflow Execution
<code>docker run -it --rm IMAGE sh</code>	Spawn ephemeral debugging interactive context nodes.
<code>docker ps -a --filter status=exited</code>	Locate terminated container configuration states.
<code>docker image prune -a -f</code>	Wipe local cache storage image footprints.
<code>docker system df -v</code>	Analyze detailed local engine disk consumption metrics.
<code>docker inspect -f '{{.NetworkSettings.IPAddress}}'</code>	Extract container target local bridge IP assignments directly.
<code>docker compose down -v</code>	Destroy application runtime infrastructure along with data volumes.
<code>docker buildx build --platform linux/amd64,linux/arm64</code>	Cross-compile multi-architecture production assets simultaneously.

APPENDIX B: MOST IMPORTANT DOCKER FLAGS

Flag Identifier	Scope Target	Operational Control Impact Description
<code>-d, --detach</code>	Runtime Engine	Detaches execution loops into isolated daemon background processes.
<code>-p, --publish</code>	Network Engine	Binds interface targets to expose system port communication vectors.
<code>-v, --volume</code>	Storage Layer	Maps data storage systems out of container runtime life cycles.
<code>--rm</code>	Lifecycle	Destroys ephemeral containers automatically on exit signals.
<code>--no-cache</code>	Build Engine	Forces reconstruction of every layer from source instructions.
<code>--restart</code>	Process Control	Sets container self-healing boundaries and retry rules.

APPENDIX C: DOCKER EXIT CODES

Exit Code	Standard Meaning	Diagnostic Assessment & Mitigation Strategy
0	Success State	Application process completed naturally without error signals.
1	Application Crash	Internal runtime error; check internal framework stack logs.
125	Docker CLI Error	Daemon processing failures; invalid parameter combinations passed.
126	Command Invoke Failure	Permissions issue or missing execution capability inside container layer.
127	Command Not Found	The target binary executable file path is missing inside image layers.
137	Fatal Error (SIGKILL)	The container was terminated by host OS (often OOM Killer due to high memory).
139	Segmentation Fault	Memory tracking layer faults; application tried writing protected locations.

APPENDIX D: DOCKER TROUBLESHOOTING QUICK REFERENCE

Scenario 1: Container instantly exits on `docker run` execution.

Verify if the image provides a persistent long-running background task execution profile or interactive processing terminal flags. Use `docker logs [id]` to extract standard stream outputs and review code stack errors.

Scenario 2: Host system is out of disk space from Docker operations.

Run `docker system df` to pinpoint the source of resource utilization. Safely clear cached assets using `docker system prune -a --volumes` after validating that all stateful components are currently active.

Scenario 3: Inter-container network communication is failing.

Use `docker network inspect` to ensure both target containers are connected to the same custom user-defined network segment, as the default bridge network does not support automatic container-to-container DNS name resolution.

APPENDIX E: COMMAND LOOKUP TABLE

Target Command Block	Functional Group	Primary Operational Objective
<code>docker run</code>	Lifecycle Management	Instantiate and start production container runtimes.
<code>docker buildx</code>	Compilation Engine	Build multi-architecture images using BuildKit engine layers.
<code>docker compose</code>	Stack Orchestration	Manage multi-container application stacks declaratively.
<code>docker network</code>	Network Routing	Configure virtual topologies and communication boundaries.
<code>docker volume</code>	State Preservation	Manage isolated persistent storage volumes for stateful workloads.
<code>docker system</code>	Resource Management	Analyze system disk space metrics and perform resource cleanups.